ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA TOÁN - TIN HỌC



Báo cáo Đồ Án Giữa Kỳ

Đề tài: Nhận dạng chữ số viết tay bằng các phương pháp phân lớp

MÔN HỌC

Nhận Dạng Mẫu

Sinh viên thực hiện:

Nguyễn Thị Huệ Giang (24110014)

Lâm Hà Quỳnh Như (24110043)

Hồ Việt Thuận Phú (24110098)

TP. Hồ Chí Minh, Ngày 5 tháng 11 năm 2026

Mục lục

1 Giới thiệu **1**

2 Dữ liệu **1**

2.1 Mô tả tập MNIST và phân tích khám phá 1

2.2 Tiền xử lý và chia tập 2

3 Phương pháp **3**

3.1 Multinomial Naive Bayes 3

3.2 Logistic Regression đa lớp (softmax) bằng Gradient Descent 4

3.3 Mạng nơ-ron nhiều lớp (MLP) 5

3.4 Ensemble: Random Forest 6

4 Thí nghiệm **7**

5 Kết quả **8**

5.1 Hiệu năng trên ba tập và đối chiếu thư viện 8

5.2 Ảnh hưởng của siêu tham số 9

5.3 Confusion matrix trên tập test 10

6 Thảo luận **10**

6.1 So sánh bốn phương pháp 10

6.2 Phân tích lỗi 12

6.3 Hạn chế 12

7 Thí nghiệm mở rộng **13**

7.1 Tập chữ số viết tay người Việt 13

7.2 Tăng cường dữ liệu và thí nghiệm độ bền 15

7.3 Giao diện demo vẽ và nhận dạng chữ số 15

8 Kết luận và hướng phát triển **16**

Tài liệu tham khảo **17**

A Phân công công việc và khai báo sử dụng AI **17**

A.1 Bảng phân công công việc và tỷ lệ đóng góp 17

A.2 Khai báo sử dụng trí tuệ nhân tạo (AI) 18

Danh sách bảng

1 Siêu tham số: không gian tìm kiếm trên validation và giá trị chọn. 7
2 Accuracy và macro-F1 của cấu hình cuối trên train (54.000), validation (6.000) và test (10.000). In đậm: tốt nhất theo cột. 8
3 Đối chiếu cài đặt tự viết với thư viện chuẩn trên tập validation. 8
4 Số lỗi của các cặp hay nhầm trên tập test (cộng cả hai chiều, ví dụ $4 \leftrightarrow 9 = 4 \to 9 + 9 \to 4$). 10
5 Recall từng lớp trên tập test. In đậm: lớp khó nhất của mỗi mô hình. 10
6 So sánh thực tiễn bốn bộ phân lớp (cấu hình cuối). 11
7 Datasheet của tập chữ số viết tay người Việt. 14
8 Accuracy trên những người viết Việt không có trong tập huấn luyện (trung bình $\pm$ độ lệch chuẩn qua 3 lần chia). 14
9 Accuracy test (%) trên ảnh gốc, xoay 30° và dịch 3 pixel. 15
10 Phân công công việc và tỷ lệ đóng góp của các thành viên. 17
11 Khai báo công cụ AI được sử dụng trong đề tài. 18

Danh sách hình vẽ

2 Accuracy validation theo siêu tham số. Thanh lỗi: độ lệch chuẩn qua 3 hạt giống. . 9
3 Confusion matrix trên 10.000 ảnh test (hàng: nhãn thật, cột: dự đoán). Tổng số lỗi: NB 1.636, LR 787, MLP 191, RF 355. 10
4 Ba mô hình giải thích được bằng hình: tham số vẽ thành ảnh 28×28. 11
5 30 ảnh test đầu tiên bị MLP đoán sai, ghi "nhãn thật → dự đoán". 12
6 Quy trình chuyển ảnh phiếu thành ảnh kiểu MNIST và tập chữ số người Việt sau xử lý. Số 1 của người Việt có nét móc ở đầu và gạch chân, số 7 có gạch ngang ở giữa, các chữ số đứng thẳng hơn MNIST. 13
7 Accuracy test theo mức biến đổi ảnh (nét đứt: có augmentation). 15
8 Demo nhận dạng chữ số vẽ tay. 16

1 Giới thiệu

Bài toán. Nhận dạng chữ số viết tay là bài toán phân lớp: xác định chữ số 0–9 từ một ảnh đầu vào. Nhóm dùng bộ dữ liệu MNIST gồm 70.000 ảnh xám 28×28 thuộc 10 lớp.

Mục tiêu. Tự cài đặt phần tính toán cốt lõi của bốn bộ phân lớp: Multinomial Naive Bayes, Logistic Regression, mạng nơ-ron nhiều lớp (MLP) và Random Forest; đối chiếu với scikit-learn và PyTorch; so sánh bằng Accuracy, Precision, Recall, F1 macro và Confusion Matrix, cùng thời gian huấn luyện, thời gian dự đoán và khả năng giải thích.

Đóng góp của nhóm.

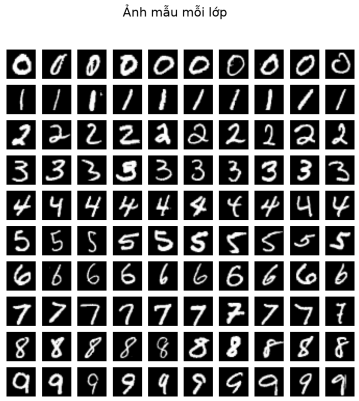
- Quy trình tiền xử lý dùng chung và bốn bộ phân lớp tự cài bằng NumPy, mỗi bộ có bằng chứng cài đúng (tính tay, kiểm tra gradient, đối chiếu thư viện).
- Khảo sát siêu tham số trên validation và so sánh bốn phương pháp trên tập test.
- Bộ 2.980 chữ số viết tay do 30 người Việt viết, dùng để đo lệch miền; thí nghiệm tăng cường dữ liệu và độ bền; chương trình demo bằng Gradio.

Mã nguồn đồ án. Toàn bộ mã nguồn và tài liệu triển khai được lưu trữ tại GitHub: `https://github.com/24110098/Nhom8_NDM_GK`.

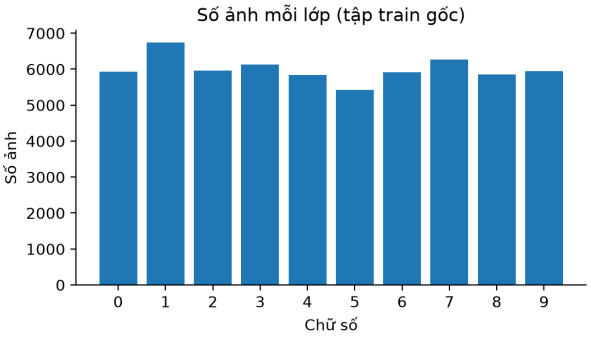
2 Dữ liệu

2.1 Mô tả tập MNIST và phân tích khám phá

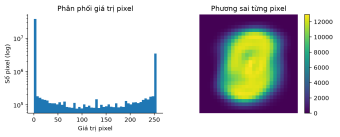
MNIST gồm 60.000 ảnh huấn luyện và 10.000 ảnh kiểm thử, ảnh xám 28×28 kiểu `uint8` (0 là nền, 255 là nét). Mỗi chữ số được thu về khung 20×20 rồi đặt vào ảnh 28×28 sao cho khối tâm nét chữ trùng tâm ảnh; người viết của hai tập không trùng nhau. Lớp `MNISTLoader` (`src/data.py`) tải bốn file gốc định dạng IDX, kiểm tra *magic number*, số mẫu và miền nhãn trước khi lưu thành `data/mnist.npz`.



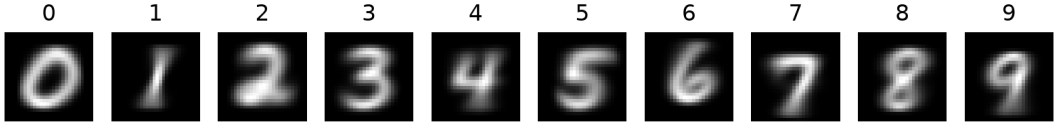
(a) Ảnh mẫu, mỗi hàng một lớp



(b) Số ảnh mỗi lớp (60.000 ảnh train gốc)



(c) Histogram pixel, phương sai pixel



(d) Ảnh trung bình của từng lớp

(e) Phân tích khám phá trên 60.000 ảnh huấn luyện gốc.

Nhận xét. (1) Nét viết trong cùng một lớp đa dạng về độ nghiêng, độ dày và kiểu viết (số 4 hở hoặc kín, số 7 có hoặc không gạch ngang). (2) Dữ liệu khá cân bằng: từ 5.421 ảnh (số 5) đến 6.742 ảnh (số 1), nên accuracy có ý nghĩa; nhóm báo cáo thêm F1 macro để mọi lớp có trọng số như nhau. (3) Ảnh rất thưa: 80,9% giá trị pixel bằng 0, histogram có hai đỉnh tại 0 và 255; 67/784 pixel ở viền luôn bằng 0 nên không mang thông tin. (4) Ảnh trung bình của 4–9 và 3–5–8 gần nhau, dự báo trước các cặp dễ nhầm.

2.2 Tiền xử lý và chia tập

Làm phẳng và chuẩn hoá. Mỗi ảnh thành vector $\mathbf{x} \in \mathbb{R}^{784}$ theo thứ tự hàng, kiểu `float32`. Cả bốn mô hình dùng chung chuẩn hoá min-max $x' = x/255 \in [0, 1]$: giữ đặc trưng không âm (điều kiện của Multinomial NB), không ước lượng tham số từ dữ liệu nên không rò rỉ thông tin từ validation/test.

Chia tập. Tập test (10.000 ảnh) giữ nguyên và chỉ dùng một lần ở cuối. Từ 60.000 ảnh huấn luyện, lớp `StratifiedSplitter` tách *phân tầng* 10% làm validation: lớp c góp $n_c/10$ mẫu, làm tròn bằng phương pháp phần dư lớn nhất để tổng đúng 6.000. Kết quả: 54.000 train / 6.000 validation / 10.000 test, hạt giống cố định 42; trung bình pixel của ba tập lần lượt là 0,131 / 0,131 / 0,133, xác nhận việc chia không làm lệch phân bố.

3 Phương pháp

Mỗi ảnh được biểu diễn bằng $\mathbf{x} \in [0,1]^D$, $D = 784$; nhãn $y \in \{0, \dots, 9\}$, $K = 10$ lớp. Cả bốn lớp mô hình có chung giao diện `fit`, `predict_proba`, `predict`.

3.1 Multinomial Naive Bayes

Mô hình. Chọn lớp có xác suất hậu nghiệm lớn nhất, với giả định các pixel độc lập có điều kiện khi biết lớp. Mô hình đa thức coi cường độ x_j là "lượng mực" tại pixel j và $\theta_{cj} = P(j \mid c)$ là tỉ lệ mực của lớp c rơi vào pixel j ($\sum_j \theta_{cj} = 1$). Với $N_{cj} = \sum_{i:y_i=c} x_{ij}$ và làm trơn Laplace hệ số α:

$$\pi_c = \frac{n_c}{n}, \qquad \theta_{cj} = \frac{N_{cj} + \alpha}{\sum_{j'} N_{cj'} + \alpha D}, \qquad s_c(\mathbf{x}) = \log \pi_c + \sum_{j=1}^{D} x_j \log \theta_{cj}, \qquad \hat{y} = \arg\max_c s_c. \tag{1}$$

Làm trơn tránh $\theta_{cj} = 0$ ở các pixel không bao giờ có mực (67 pixel viền); tính trong miền log tránh tràn dưới khi nhân 784 xác suất nhỏ. Xác suất hậu nghiệm là softmax(s), trừ $\max_k s_k$ trước khi lấy mũ.

Algorithm 1 Multinomial Naive Bayes

function FIT($X \in \mathbb{R}^{n \times D}$, $\mathbf{y}$, α)
  for mỗi lớp c **do**
    $\log \pi_c \leftarrow \log(n_c/n)$; $\mathbf{N}_c \leftarrow$ tổng các hàng của X có nhãn c
    $\log \boldsymbol{\theta}_c \leftarrow \log\big((\mathbf{N}_c + \alpha)/(\sum \mathbf{N}_c + \alpha D)\big)$
  end for
end function
function PREDICT(X)
  $S \leftarrow X\,(\log \Theta)^\top + \log \boldsymbol{\pi}$ ▷ một phép nhân ma trận, $S \in \mathbb{R}^{n \times K}$
  return arg max theo từng hàng của S
end function

Cài đặt và kiểm chứng (`naive_bayes.py`). Huấn luyện là một lượt cộng dồn $O(nD)$; dữ liệu được ép về `float64` để tổng N_{cj} trên hàng nghìn ảnh không bị sai số làm tròn. Vì công thức xác định, nhóm yêu cầu khớp *tuyệt đối* với `MultinomialNB` của scikit-learn: `log_feature_prob`, `log_prior` và `predict_proba` trùng nhau theo `np.allclose`, dự đoán trùng 100%. Tham số $\boldsymbol{\theta}_c$ vẽ thành ảnh cho đúng hình dạng từng chữ số (Hình 4). Siêu tham số làm trơn được chọn là $\alpha = 0{,}001$ (dùng chung cho cả cài đặt của nhóm và `MultinomialNB` khi đối chiếu); giá trị này đủ nhỏ để tránh $\log 0$ mà gần như không làm lệch ước lượng θ_{cj}.

3.2 Logistic Regression đa lớp (softmax) bằng Gradient Descent

Mô hình. Mỗi lớp có một vector trọng số, hình dung như một "tấm khuôn" đo mức phù hợp của ảnh với chữ số đó. Với $W \in \mathbb{R}^{784\times 10}$, $\mathbf{b} \in \mathbb{R}^{10}$, nhãn one-hot Y, hàm mất mát cross-entropy có điều chuẩn L_2:

$$Z = XW + \mathbf{b}, \qquad P_{ik} = \frac{e^{Z_{ik}}}{\sum_j e^{Z_{ij}}}, \qquad \mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n} \log P_{i,y_i} + \frac{\lambda}{2}\|W\|_F^2. \tag{2}$$

Với mini-batch B mẫu, gradient có dạng gọn "dự đoán trừ đáp án":

$$G = \frac{P - Y}{B}, \qquad \nabla_W = X^\top G + \lambda W, \qquad \nabla_{\mathbf{b}} = \textstyle\sum_i G_i, \qquad W \leftarrow W - \eta\nabla_W, \quad \mathbf{b} \leftarrow \mathbf{b} - \eta\nabla_{\mathbf{b}}. \tag{3}$$

Algorithm 2 Huấn luyện Logistic Regression bằng mini-batch Gradient Descent

function FIT($X, \mathbf{y}, \lambda, \eta, E, B$)
  Khởi tạo $W \sim \mathcal{N}(0, 0{,}01^2)$, $\mathbf{b} \leftarrow \mathbf{0}$
  for $e = 1, \ldots, E$ **do**
    for mỗi mini-batch (X_I, Y_I) sau khi xáo trộn **do**
      $P \leftarrow \mathrm{softmax}(X_I W + \mathbf{b})$; $G \leftarrow (P - Y_I)/B$
      $W \leftarrow W - \eta\,(X_I^\top G + \lambda W)$; $\mathbf{b} \leftarrow \mathbf{b} - \eta \sum_i G_i$
    end for
    Ghi nhận train loss và validation accuracy
  end for
end function

Cài đặt và kiểm chứng (`logistic.py`). Softmax trừ giá trị lớn nhất trên mỗi hàng trước khi lấy hàm mũ nhằm hạn chế tràn số. Các siêu tham số được thiết lập gồm learning rate $\eta = 0.1$, hệ số chính quy $\lambda = 10^{-4}$, số epoch là 20 và batch size là 128. Trong quá trình lựa chọn siêu tham số, $\eta = 0.1$ được chọn là learning rate phù hợp nhất cho chương trình. (1) *Kiểm tra gradient* bằng sai phân trung tâm trên 20 ảnh, với $\lambda = 10^{-3}$ và kiểu dữ liệu `float64`: sai số tương đối lớn nhất là $4{,}90 \times 10^{-7} < 10^{-6}$. (2) *Huấn luyện*: train loss giảm đều từ 0,408 xuống 0,277 sau 20 epoch. (3) *Đối chiếu* với `LogisticRegression` của scikit-learn sau khi quy đổi $C = 1/(n\lambda)$ (Bảng 3).

3.3 Mạng nơ-ron nhiều lớp (MLP)

Lan truyền xuôi. Mạng có L lớp với kích thước $d_0 = 784, d_1, \ldots, d_L = 10$. Đặt $\mathbf{A}_0 = \mathbf{X}$:

$$\mathbf{Z}_l = \mathbf{A}_{l-1}\mathbf{W}_l + \mathbf{b}_l, \qquad \mathbf{A}_l = \mathrm{ReLU}(\mathbf{Z}_l)\ (l < L), \qquad \mathbf{P} = \mathbf{A}_L = \mathrm{softmax}(\mathbf{Z}_L). \tag{4}$$

Hàm mục tiêu là cross-entropy cộng $\frac{\lambda}{2}\sum_l \|\mathbf{W}_l\|_F^2$ (trong thí nghiệm $\lambda = 0$, chống khớp quá mức bằng dừng sớm).

Lan truyền ngược. Với softmax kết hợp cross-entropy, $\partial J/\partial \mathbf{Z}_L = (\mathbf{P} - \mathbf{Y})/n$. Đặt $\mathbf{\Delta}_l = \partial J/\partial \mathbf{Z}_l$:

$$\mathbf{\Delta}_L = \tfrac{1}{n}(\mathbf{P} - \mathbf{Y}), \qquad \frac{\partial J}{\partial \mathbf{W}_l} = \mathbf{A}_{l-1}^\top \mathbf{\Delta}_l + \lambda \mathbf{W}_l, \qquad \frac{\partial J}{\partial \mathbf{b}_l} = \mathbf{\Delta}_l^\top \mathbf{1}, \qquad \mathbf{\Delta}_{l-1} = \left(\mathbf{\Delta}_l \mathbf{W}_l^\top\right) \odot \mathbb{I}[\mathbf{Z}_{l-1} > 0]. \tag{5}$$

Tối ưu. Khởi tạo He $\mathbf{W}_l \sim \mathcal{N}(0, 2/d_{l-1})$ giữ phương sai kích hoạt ổn định qua các lớp ReLU. SGD với momentum: $\mathbf{v} \leftarrow \mu\mathbf{v} - \eta\mathbf{g}$, $\theta \leftarrow \theta + \mathbf{v}$. Dừng sớm khi accuracy validation không tăng sau `patience` epoch và khôi phục trọng số tốt nhất.

Algorithm 3 Huấn luyện MLP bằng mini-batch SGD + momentum và dừng sớm

Khởi tạo He cho $\mathbf{W}_l$; $\mathbf{b}_l, \mathbf{v}_l \leftarrow \mathbf{0}$; $best \leftarrow -1$; $wait \leftarrow 0$
for $e = 1$ **to** E **do**
  for mỗi mini-batch $\mathcal{B}$ sau khi xáo trộn **do**
    $\mathbf{P} \leftarrow$ lan truyền xuôi (4), lưu $\mathbf{A}_l, \mathbf{Z}_l$
    Tính $\partial J/\partial \mathbf{W}_l, \partial J/\partial \mathbf{b}_l$ từ $l = L$ về 1 theo (5)
    Cập nhật mọi tham số bằng SGD + momentum
  end for
  $acc \leftarrow$ accuracy validation
  if $acc > best$ **then** $best \leftarrow acc$; $wait \leftarrow 0$; lưu trọng số
  else $wait \leftarrow wait + 1$; **if** $wait \geq p$ **then break**
  end if
end for
Khôi phục trọng số đã lưu

Cài đặt (`mlp.py`, NumPy). Mọi phép tính vector hoá trên cả mini-batch; `predict_proba` chạy theo lô 4.096 mẫu, không lưu cache; tự dừng khi loss là NaN. Cấu hình: $\eta = 0{,}05$, $\mu = 0{,}9$, batch 128, tối đa 15 epoch, `patience` = 3.

Kiểm chứng. (1) *Tính tay*: mạng 2–2–2 với trọng số cho trước cho $\mathbf{P} = (0{,}4477; 0{,}5523)$ và gradient trùng kết quả tính tay đến 4 chữ số. (2) *Kiểm tra gradient* trên mạng $(20, 16, 12, 4)$, $\lambda = 10^{-3}$, `float64`: sai số tương đối lớn nhất $8{,}8 \times 10^{-8}$. (3) *Khớp số học với scikit-learn*: chép

trọng số sang `MLPClassifier` (quy đổi $\alpha = \lambda B$, tắt Nesterov), xác suất dự đoán trùng tuyệt đối, sau 5 bước SGD sai lệch trọng số lớn nhất $1{,}1 \times 10^{-16}$. (4) *Đối chiếu PyTorch* khi huấn luyện đầy đủ cùng kiến trúc $(784, 128, 10)$ (Bảng 3).

3.4 Ensemble: Random Forest

Cây CART với tiêu chí Gini. Mỗi nút hỏi "$x_f \le t$?". Phép tách (f, t) được chọn để độ giảm Gini lớn nhất:

$$G = 1 - \sum_{k=1}^{K} p_k^2, \qquad \Delta(f,t) = G_{\text{cha}} - \Big(\tfrac{n_L}{n} G_L + \tfrac{n_R}{n} G_R\Big). \tag{6}$$

Cây phát triển đệ quy tới độ sâu tối đa hoặc nút thuần nhất; mỗi lá lưu vector tỉ lệ lớp $\mathbf{p}_{\text{lá}}$.

Hai nguồn ngẫu nhiên. Mỗi cây học trên một mẫu *bootstrap* (lấy n mẫu có hoàn lại; mỗi mẫu bị bỏ sót với xác suất $\approx e^{-1} \approx 0{,}368$), và tại mỗi nút chỉ xét $m = \lfloor \sqrt{D} \rfloor = 28$ đặc trưng ngẫu nhiên. Hai nguồn này giảm tương quan ρ giữa các cây; phương sai của trung bình T cây là $\rho\sigma^2 + \frac{1-\rho}{T}\sigma^2$. Dự đoán bằng *soft voting* $P(c \mid \mathbf{x}) = \frac{1}{T}\sum_b \mathbf{p}^{(b)}_{\text{lá}(\mathbf{x})}[c]$. Mẫu không nằm trong bootstrap của cây b là mẫu *out-of-bag* (OOB); dự đoán mỗi mẫu bằng các cây không thấy nó cho *OOB score*, một ước lượng accuracy không cần tập riêng. Độ quan trọng đặc trưng là tổng $n_{\text{nút}} \cdot \Delta$ qua các nút tách theo đặc trưng đó.

Algorithm 4 Tìm phép tách tốt nhất tại một nút (sắp xếp + tổng cộng dồn)

function BestSplit(X, $\mathbf{y}$, tập đặc trưng ngẫu nhiên F)
  $\Delta^* \leftarrow 0$; $\mathbf{n}_L \leftarrow (1, \dots, n-1)$; $\mathbf{n}_R \leftarrow n - \mathbf{n}_L$
  for mỗi $f \in F$ **do**
    Sắp xếp mẫu theo $x_f \to \mathbf{x}^s, \mathbf{y}^s$; $H \leftarrow \text{onehot}(\mathbf{y}^s)$
    $C_L \leftarrow \text{cumsum}(H)$ bỏ hàng cuối; $C_R \leftarrow \sum H - C_L$ ▷ đếm lớp hai bên cho mọi vị trí
    $\mathbf{g} \leftarrow \big(\mathbf{n}_L\, G(C_L/\mathbf{n}_L) + \mathbf{n}_R\, G(C_R/\mathbf{n}_R)\big)/n$ ▷ Gini có trọng số
    Loại vị trí nằm giữa hai giá trị bằng nhau ($x^s_j = x^s_{j+1}$)
    $j^* \leftarrow \arg\min_j g_j$; **if** $G_{\text{cha}} - g_{j^*} > \Delta^*$ **then** lưu $\big(f,\ (x^s_{j^*} + x^s_{j^*+1})/2\big)$
  end for
  return phép tách tốt nhất (hoặc rỗng: nút thành lá)
end function

Cài đặt (`random_forest.py`, NumPy). Mô hình Random Forest sử dụng `n_estimators = 50` và `max_depth = 15`. Thuật toán 4 tìm ngưỡng *chính xác* (không chia bin): sắp xếp một lần $O(n \log n)$, rồi `np.cumsum` cho số đếm lớp của mọi vị trí tách trong một lượt, thay vì $O(n^2)$ khi thử từng ngưỡng. Nhãn được mã hoá về $0..K{-}1$ và K truyền cho mọi cây, nên vector xác suất của lá luôn dài K.

Mỗi cây có bộ sinh ngẫu nhiên riêng rút từ `random_state` nên kết quả tái lập được.

Kiểm chứng. (1) *Tính tay*: trên ví dụ 10 ảnh một đặc trưng (nhãn $0, 0, 0, 0, 1, 1, 7, 1, 7, 7$), ngưỡng $t = 0{,}40$ cho $\Delta = 0{,}360$ lớn nhất, Gini của cả 9 vị trí khớp bảng tính tay. (2) *Dữ liệu tổng hợp* 2 đặc trưng có ích, 6 đặc trưng nhiễu: importance $\approx 0{,}30$ và $0{,}34$ cho đặc trưng có ích, $\approx 0{,}06$ cho mỗi đặc trưng nhiễu. (3) *Đối chiếu* `RandomForestClassifier` cùng tham số (Bảng 3). (4) *Nhất quán nội tại*: OOB = 0,9549 gần accuracy validation 0,9622; 158/784 pixel (gồm 67 pixel luôn bằng 0) không bao giờ được dùng để tách (Hình 4).

4 Thí nghiệm

Môi trường. Python 3.14.5, NumPy 2.5.4, SciPy 1.18.1, pandas 3.0.6, Matplotlib 3.11.2 trên Windows; CPU/RAM: 16GB. scikit-learn 1.9.1 và PyTorch 2.14.1+cpu chỉ dùng để đối chiếu. Toàn bộ kết quả tái tạo bằng *Restart & Run All* trên `main.ipynb`.

Quy trình chọn siêu tham số. Mọi siêu tham số được chọn trên validation; tập test chỉ đánh giá **một lần** sau khi chốt cấu hình, mỗi mô hình được huấn luyện lại trên 54.000 ảnh. Mô hình có ngẫu nhiên khi huấn luyện (Logistic Regression, MLP) chạy 3 hạt giống $\{0, 1, 2\}$ và so sánh trung bình $\pm$ độ lệch chuẩn. Logistic Regression cố định $\lambda = 10^{-4}$, 20 epoch, batch 128. Random Forest cố định `max_depth`= 15, `max_features`= $\sqrt{D}$, soft voting (một cây mất khoảng 9 s huấn luyện); đường accuracy theo số cây lấy từ *một* rừng 50 cây bằng cách cộng dồn xác suất của k cây đầu. Thước đo: accuracy và F1 macro trên ba tập, confusion matrix trên test, thời gian dự đoán mỗi ảnh.

Bảng 1: Siêu tham số: không gian tìm kiếm trên validation và giá trị chọn.

Mô hình	Siêu tham số quét	Giá trị thử	Chọn
Multinomial NB	α	$10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100$	10^{-3}
Logistic Regression	learning rate η ($\times$3 seed)	$10^{-3}, 10^{-2}, 10^{-1}, 1$	0.1
MLP	kiến trúc ($\times$3 seed)	784-32-10, 784-128-10, 784-512-10, 784-256-128-10	784-256-128-10
Random Forest	số cây T	$1, 2, 5, 10, 20, 30, 50$	50

5 Kết quả

5.1 Hiệu năng trên ba tập và đối chiếu thư viện

Bảng 2: Accuracy và macro-F1 của cấu hình cuối trên train (54.000), validation (6.000) và test (10.000). In đậm: tốt nhất theo cột.

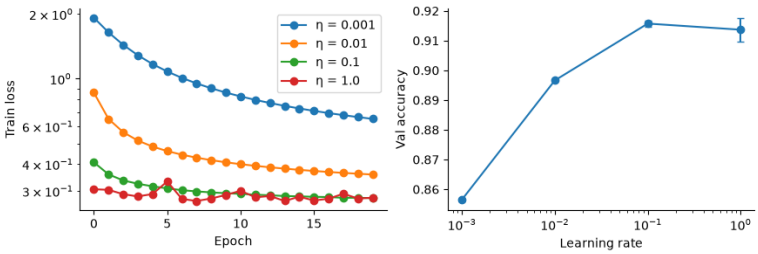
	Accuracy			Macro-F1			Khoảng cách	Dự đoán
Mô hình	Train	Val	Test	Train	Val	Test	train−test	(ms/ảnh)
Multinomial NB	0.8248	0.8210	0.8364	0.8235	0.8199	0.8349	−0.012	0.007
Logistic Regression	0.9244	0.9167	0.9213	0.9234	0.9155	0.9201	0.003	0.002
MLP 784-256-128-10	**0.9998**	**0.9813**	**0.9809**	**0.9998**	**0.9813**	**0.9808**	0.019	0.002
Random Forest (T=50)	0.9971	0.9622	0.9645	0.9971	0.9618	0.9642	0.033	0.206

Bảng 3: Đối chiếu cài đặt tự viết với thư viện chuẩn trên tập validation.

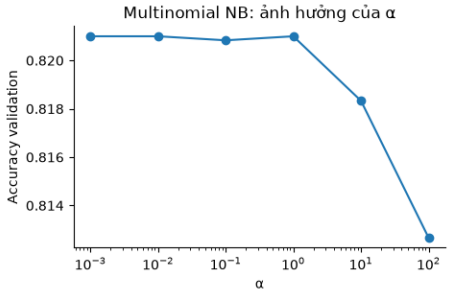
Mô hình	Đối chiếu với	Tự cài	Thư viện	Tiêu chí / nhận xét
Multinomial NB	sklearn `MultinomialNB`	0.8210	0.8210	tham số và xác suất khớp; dự đoán trùng 100%
Logistic Regression	sklearn `LogisticRegression`	0.9167	0.9183	quy đổi $C = 1/(n\lambda)$; chênh 0.16 điểm
MLP 784-128-10	PyTorch (SGD momentum)	0.9768	0.9775	cùng kiến trúc, khởi tạo He; chênh 0.07 điểm
Random Forest (T=50)	sklearn `RandomForestClassifier`	0.9622	0.9608	cùng tham số; chênh 0.14 điểm

Thứ hạng nhất quán trên cả ba tập: MLP > Random Forest > Logistic Regression > Naive Bayes. Accuracy validation và test chênh nhau dưới 1.6 điểm với mọi mô hình, nên validation 6.000 ảnh là thước đo đáng tin để chọn siêu tham số. Naive Bayes khớp tuyệt đối với scikit-learn; ba mô hình còn lại có thành phần ngẫu nhiên hoặc bộ tối ưu khác nhau và đều chênh dưới 0.2 điểm, xác nhận cài đặt đúng.

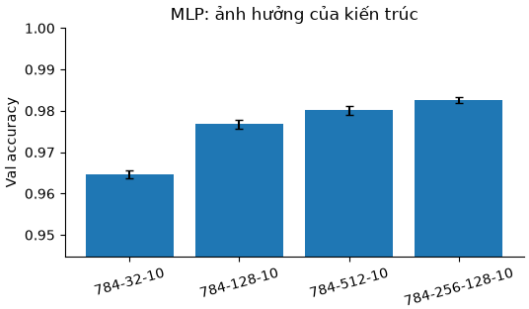
5.2 Ảnh hưởng của siêu tham số



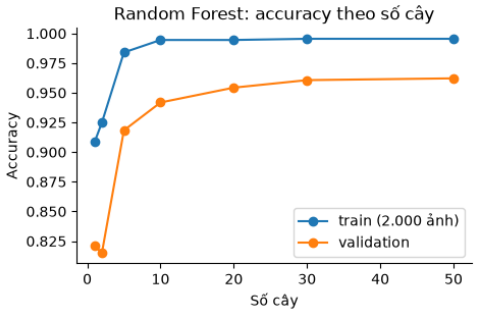
(a) Logistic Regression: loss theo epoch và val accuracy theo η.



(b) Multinomial NB: hệ số làm trơn α.



(c) MLP: kiến trúc (3 seed).

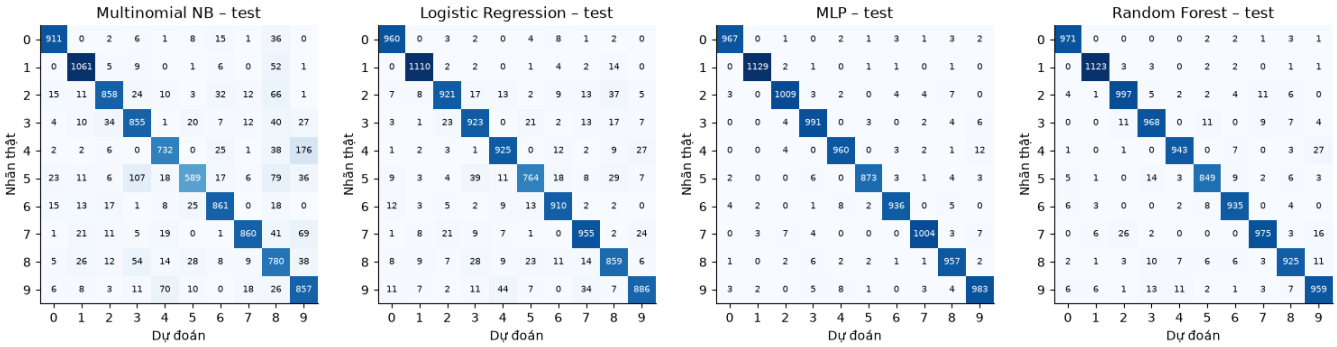


(d) Random Forest: accuracy theo số cây.

Hình 2: Accuracy validation theo siêu tham số. Thanh lỗi: độ lệch chuẩn qua 3 hạt giống.

- **Logistic Regression**: $\eta = 10^{-3}$ hội tụ quá chậm (0.856 sau 20 epoch); $\eta = 0.1$ cao nhất (0.916) và ổn định; $\eta = 1$ có trung bình tương đương nhưng loss dao động và độ lệch giữa các seed lớn hơn.
- **NB**: accuracy gần như phẳng (≈ 0.821) với $\alpha \leq 1$ vì tổng mực mỗi lớp rất lớn, rồi giảm còn 0.813 ở $\alpha = 100$ do mọi θ_{cj} bị kéo về phân phối đều.
- **MLP**: accuracy tăng theo dung lượng ($0.965 \to 0.977 \to 0.980 \to 0.983$); mạng hai lớp ẩn tốt nhất dù ít tham số hơn mạng 784-512-10 (235.146 so với 407.050).
- **Random Forest**: bão hoà quanh 0.96 từ 30 cây; train đạt ≈ 1.0 từ 10 cây, khoảng cách train–validation không thu hẹp khi thêm cây vì thêm cây chỉ giảm phương sai.

5.3 Confusion matrix trên tập test



Hình 3: Confusion matrix trên 10.000 ảnh test (hàng: nhãn thật, cột: dự đoán). Tổng số lỗi: NB 1.636, LR 787, MLP 191, RF 355.

Bảng 4: Số lỗi của các cặp hay nhầm trên tập test (cộng cả hai chiều, ví dụ $4 \leftrightarrow 9 = 4 \to 9 + 9 \to 4$).

Mô hình	$4 \leftrightarrow 9$	$3 \leftrightarrow 5$	$7 \leftrightarrow 9$	$2 \leftrightarrow 7$	$2 \to 8$	Tổng lỗi
Multinomial NB	246 (176+70)	127 (20+107)	87	23	66	1.636
Logistic Regression	71 (27+44)	60 (21+39)	58	34	37	787
MLP	20 (12+8)	9 (3+6)	10	11	7	191
Random Forest	38 (27+11)	25 (11+14)	19	37 (11+26)	6	355

Bảng 5: Recall từng lớp trên tập test. In đậm: lớp khó nhất của mỗi mô hình.

Mô hình	0	1	2	3	4	5	6	7	8	9
Multinomial NB	0.930	0.935	0.831	0.847	0.745	**0.660**	0.899	0.837	0.801	0.849
Logistic Regression	0.980	0.978	0.892	0.914	0.942	**0.857**	0.950	0.929	0.882	0.878
MLP	0.987	0.995	0.978	0.981	0.978	0.979	0.977	0.977	0.983	**0.974**
Random Forest	0.991	0.989	0.966	0.958	0.960	0.952	0.976	**0.948**	0.950	0.950

Chữ số 0 và 1 dễ nhất (recall ≥ 0.93 ở mọi mô hình); cặp $4 \leftrightarrow 9$ là nguồn lỗi chung lớn nhất. Recall của MLP đồng đều nhất giữa các lớp (0.974–0.995).

6 Thảo luận

6.1 So sánh bốn phương pháp

Độ chính xác và thiên kiến mô hình. Naive Bayes và Logistic Regression đều có biên quyết định *tuyến tính* trong không gian pixel (điểm NB $s_c = \log \pi_c + \mathbf{x}^\top \log \boldsymbol{\theta}_c$ cũng tuyến tính theo $\mathbf{x}$).

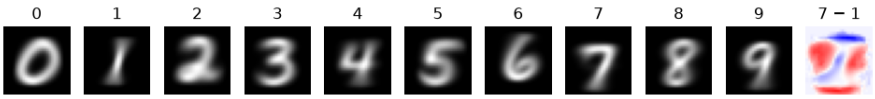
Khác biệt nằm ở cách tìm trọng số: NB *đếm* theo giả định pixel độc lập, còn LR *tối ưu* trực tiếp cross-entropy; cùng một họ biên, LR giảm lỗi test từ 1.636 xuống 787. Cả hai có khoảng cách train–test gần 0, tức *underfit*. MLP và Random Forest có biên phi tuyến, đạt 96–98%, đổi lại khoảng cách train–test lớn hơn (1.9 và 3.3 điểm).

MLP so với Random Forest. Cả hai gần như thuộc lòng tập train, nhưng MLP tổng quát hoá tốt hơn (191 so với 355 lỗi). Mỗi nút cây chỉ hỏi *một* pixel so với một ngưỡng, nên biên gồm các mặt cắt song song trục; nét bút dịch 1–2 pixel làm thay đổi hàng loạt câu trả lời. Nơ-ron ẩn của MLP là tổ hợp tuyến tính của mọi pixel, học được "mẫu nét" bền hơn với biến dạng nhỏ.

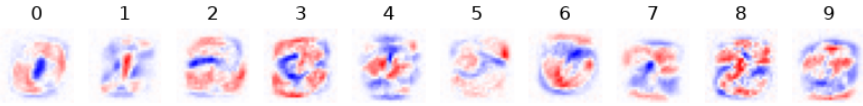
Chi phí và khả năng giải thích.

Bảng 6: So sánh thực tiễn bốn bộ phân lớp (cấu hình cuối).

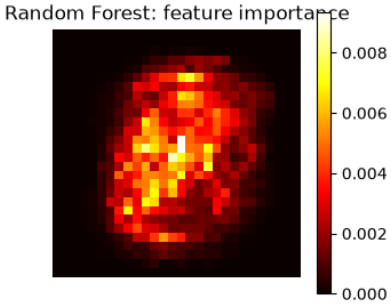
Mô hình	Test acc.	Dự đoán (ms/ảnh)	Kích thước file	Diễn giải
Multinomial NB	0.836	0.007	0.1 MB	bản đồ θ_c
Logistic Regression	0.921	0.002	0.1 MB	trọng số mỗi lớp vẽ thành ảnh
MLP	0.981	0.002	1.9 MB	khó, chỉ gián tiếp
Random Forest	0.965	0.206	35.1 MB	feature importance từng pixel



(a) Multinomial NB: $\boldsymbol{\theta}_c$ của 10 lớp và hiệu $\log \boldsymbol{\theta}_7 - \log \boldsymbol{\theta}_1$.



(b) Logistic Regression: trọng số của 10 lớp.



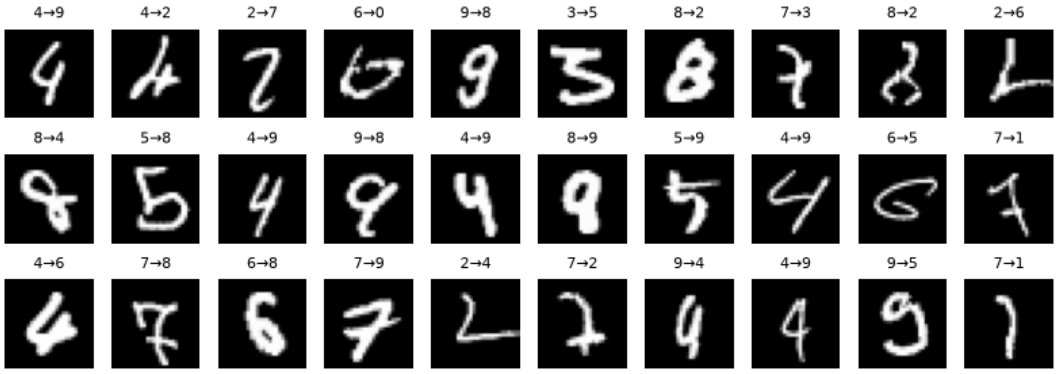
(c) Random Forest: feature importance.

Hình 4: Ba mô hình giải thích được bằng hình: tham số vẽ thành ảnh 28×28.

Naive Bayes huấn luyện tức thì và là đường cơ sở tốt; $\boldsymbol{\theta}_c$ là các chữ số mờ đúng hình dạng, và bản đồ $7 - 1$ cho thấy pixel nào "bỏ phiếu" cho mỗi lớp. Random Forest không cần chuẩn hoá hay learning rate, có OOB score miễn phí và importance tập trung ở vùng tâm, nhưng là mô hình nặng nhất (35 MB) và dự đoán chậm hơn MLP khoảng 100 lần do duyệt cây bằng vòng lặp Python. MLP chính xác nhất với chi phí dự đoán ngang mô hình tuyến tính, nhưng khó diễn giải nhất.

6.2 Phân tích lỗi

Các cặp nhầm có nguyên nhân hình học. Các cặp hay nhầm giống nhau giữa các mô hình, chỉ khác độ lớn (Bảng 4). $4 \leftrightarrow 9$ khó nhất ở mọi mô hình: chữ 4 viết "đóng" phần trên gần như trùng chữ 9. $3 \leftrightarrow 5$ và $7 \leftrightarrow 9$ chia sẻ phần lớn nét. Điều này khớp với EDA: ảnh trung bình của các cặp này giống nhau nhất. Riêng Naive Bayes có lỗi kiểu "hút về một lớp": lớp có bản đồ mực rộng (8, 9) nhận điểm cao cho nhiều ảnh lạ (cột 8 nhận 396 ảnh sai), và chữ 5 bị đoán thành 3 tới 107 lần.



Hình 5: 30 ảnh test đầu tiên bị MLP đoán sai, ghi "nhãn thật → dự đoán".

Lỗi còn lại của MLP phần lớn là ảnh thật sự mơ hồ (Hình 5): chữ 4 đỉnh khép kín bị đọc là 9, chữ 2 viết như chữ L, chữ 7 có gạch ngang đọc thành 2 hoặc 3; một số ảnh có thể bị gán nhãn sai từ gốc.

6.3 Hạn chế

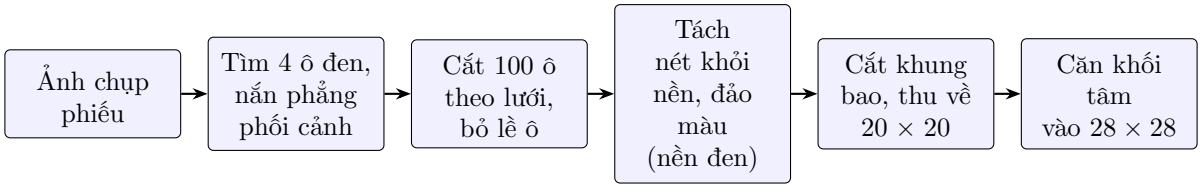
- **Tìm kiếm siêu tham số hạn chế**: mỗi mô hình chỉ quét một siêu tham số chính; Random Forest cố định `max_depth` vì chi phí, MLP chưa quét learning rate và λ. Accuracy test lấy từ một lần huấn luyện; sai số chuẩn khoảng 0.13–0.37 điểm, nên chênh lệch nhỏ khi đối chiếu thư viện nằm trong biên độ ngẫu nhiên.
- **Hiệu năng Random Forest**: dự đoán từng ảnh bằng Python (0.2 ms/ảnh) và huấn luyện tuần tự (≈7.5 phút cho 50 cây); có thể lưu cây dạng mảng và huấn luyện song song.
- **Biểu diễn đầu vào thô**: cả bốn mô hình xem ảnh như 784 pixel độc lập vị trí, nên rất nhạy với dịch chuyển (mục 7.2) và với lệch miền (mục 7.1).

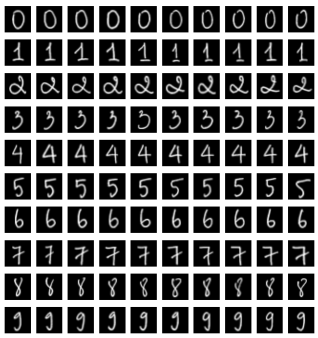
- **Giả định của Naive Bayes**: coi cường độ là "số đếm" và pixel độc lập đều không đúng với ảnh; đây là giới hạn của mô hình, không phải của cài đặt.

7 Thí nghiệm mở rộng

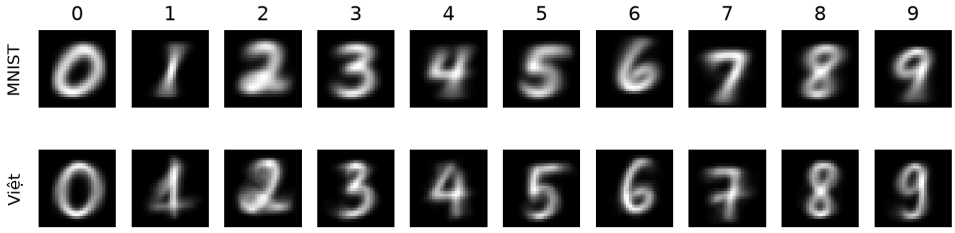
7.1 Tập chữ số viết tay người Việt

Thu thập và xử lý. Mỗi người viết điền một phiếu A4 lưới 10×10 ô (mỗi hàng một chữ số, viết 10 lần một cách tự nhiên) và ký đồng ý cho sử dụng dữ liệu; bốn ô đen ở góc phiếu làm dấu định vị. Nhóm thu 30 phiếu của 30 người (mã W01–W30), chụp bằng điện thoại. Module `vn_digits.py` chuyển ảnh phiếu sang định dạng MNIST (Hình 6); bước cuối lặp lại đúng quy trình tạo MNIST, nên khác biệt còn lại chủ yếu là nét viết chứ không phải định dạng. Sau khi xem lại từng phiếu, nhóm loại 20 ô viết nhầm, gạch xoá hoặc bẩn, còn **2.980 ảnh** (294–300 ảnh mỗi lớp).





(a) 100 ô của phiếu W01



(b) Ảnh trung bình: MNIST (trên) và tập Việt (dưới)

Hình 6: Quy trình chuyển ảnh phiếu thành ảnh kiểu MNIST và tập chữ số người Việt sau xử lý. Số 1 của người Việt có nét móc ở đầu và gạch chân, số 7 có gạch ngang ở giữa, các chữ số đứng thẳng hơn MNIST.

Bảng 7: Datasheet của tập chữ số viết tay người Việt.

Mục đích	Đo mức lệch miền giữa MNIST và chữ số người Việt; đánh giá augmentation và việc bổ sung ít dữ liệu đúng miền.
Thành phần	2.980 ảnh xám 28×28, nhãn 0–9, mã người viết W01–W30; khoảng 100 ảnh mỗi người.
Thu thập	Phiếu A4 có 4 ô định vị; bút bi và bút nước; chụp điện thoại.
Đồng ý & riêng tư	Người viết ký xác nhận; dữ liệu công bố chỉ có mã người viết, không có tên hay chữ ký.
Hạn chế	30 người chưa đại diện cho người Việt nói chung; 10 lần viết liên tiếp của một người rất giống nhau.

Thiết kế thí nghiệm. Ba thí nghiệm dùng nguyên cấu hình đã chọn trên validation MNIST, *không* chỉnh lại theo tập Việt: **(a)** huấn luyện trên MNIST; **(b)** MNIST thêm augmentation (mục 7.2); **(c)** MNIST cộng ảnh của một phần người Việt (lặp 5 lần để không bị 54.000 ảnh MNIST lấn át). Tập Việt được chia *theo người viết* (30% người viết làm tập kiểm tra), để toàn bộ ảnh của một người chỉ nằm ở một phía; chia ngẫu nhiên theo ảnh sẽ cho kết quả lạc quan giả tạo vì mô hình được kiểm tra trên nét chữ đã thấy. Cả ba thí nghiệm đánh giá trên cùng nhóm người viết kiểm tra, lặp 3 lần chia.

Bảng 8: Accuracy trên những người viết Việt không có trong tập huấn luyện (trung bình ± độ lệch chuẩn qua 3 lần chia).

Mô hình	MNIST val	(a) MNIST	(b) + augmentation	(c) + dữ liệu Việt	Giảm ở (a)
Multinomial NB	0.8210	0.5751 ± 0.018	0.5624 ± 0.017	0.6279 ± 0.019	24.6 điểm
Logistic Regression	0.9167	0.6904 ± 0.045	0.6357 ± 0.036	0.7844 ± 0.041	22.6 điểm
MLP	**0.9813**	**0.8935 ± 0.026**	**0.9029 ± 0.027**	**0.9456 ± 0.024**	8.8 điểm
Random Forest	0.9622	0.6879 ± 0.025	0.7703 ± 0.019	0.8789 ± 0.035	27.4 điểm

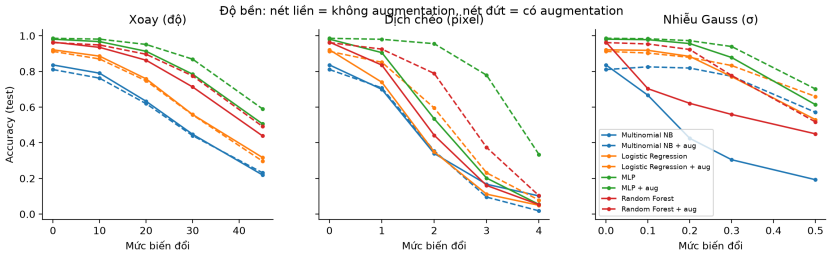
Nhận xét. Lệch miền làm cả bốn mô hình giảm mạnh (Bảng 8); MLP giảm ít nhất (8.8 điểm), Random Forest giảm nhiều nhất (27.4 điểm), nhất quán với độ nhạy theo vị trí pixel đã nêu ở mục Thảo luận. Augmentation (b) chỉ giúp MLP và Random Forest, còn làm giảm hai mô hình tuyến tính: xoay, dịch, nhiễu không tạo ra được các *kiểu viết* mới như gạch ngang ở số 7 hay móc ở số 1. Thêm một ít dữ liệu đúng miền (c) hiệu quả hơn hẳn với mọi mô hình: MLP lên 0.946, Random Forest lên 0.879.

7.2 Tăng cường dữ liệu và thí nghiệm độ bền

Hàm `augment_dataset` thêm cho mỗi ảnh train một bản biến đổi ngẫu nhiên (xoay $\pm 15°$, dịch ± 2 pixel, nhiễu Gauss $\sigma = 0.1$) và ghép với dữ liệu gốc: $X_{\text{aug}} = [X_{\text{tr}}; X_{\text{trans}}]$, $y_{\text{aug}} = [y_{\text{tr}}; y_{\text{tr}}]$ (108.000 ảnh). Hàm `perturb` làm khó tập test: xoay $0°$–$45°$, dịch chéo 0–4 pixel, nhiễu $\sigma \in \{0, 0.1, 0.2, 0.3, 0.5\}$. Mỗi mô hình được huấn luyện trên dữ liệu gốc và dữ liệu tăng cường với cùng siêu tham số.

Bảng 9: Accuracy test (%) trên ảnh gốc, xoay 30° và dịch 3 pixel.

Mô hình	Gốc	Xoay	Dịch
NB	83.64	44.75	16.72
NB + aug	81.06	43.99	9.53
LR	92.13	55.83	11.17
LR + aug	91.24	55.62	23.13
MLP	98.09	78.44	20.19
MLP + aug	**98.58**	**86.88**	**77.91**
RF	96.45	71.29	16.05
RF + aug	96.18	77.49	37.39

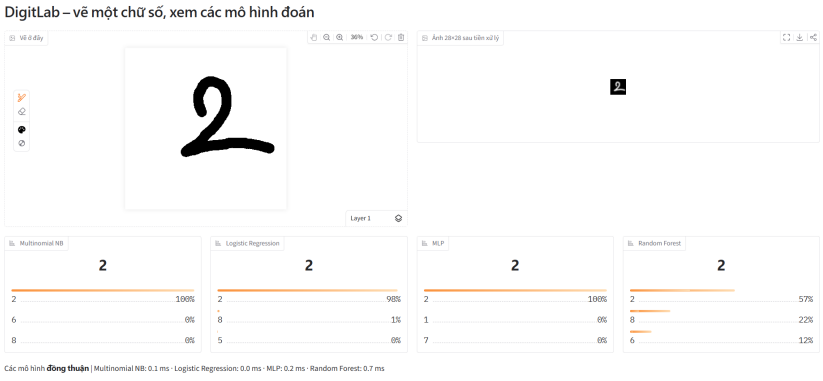


Hình 7: Accuracy test theo mức biến đổi ảnh (nét đứt: có augmentation).

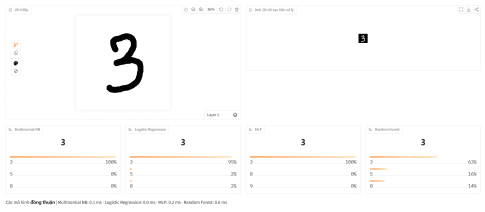
Nhận xét. Không augmentation, chỉ dịch 3 pixel đã đưa accuracy của cả bốn mô hình xuống 11–20%: biểu diễn pixel thô không bất biến với tịnh tiến. Augmentation giúp MLP rõ rệt (dịch 3 pixel: 20.19% → 77.91%; xoay 30°: 78.44% → 86.88%) và giúp Random Forest một phần, nhưng gần như không giúp Logistic Regression và còn làm giảm Naive Bayes: mô hình tuyến tính không đủ dung lượng để học đồng thời ảnh gốc và ảnh biến đổi. Hiệu quả của augmentation vì vậy phụ thuộc vào năng lực mô hình.

7.3 Giao diện demo vẽ và nhận dạng chữ số

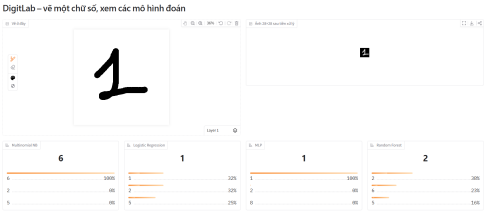
Demo viết bằng Gradio (`app.py`) nạp bốn mô hình đã lưu (`results/models/*.pkl`). Người dùng vẽ một chữ số; ảnh được chuyển sang mức xám, làm sạch và đưa về 28×28 bằng *chính* hàm tiền xử lý của tập chữ số Việt, rồi đưa vào bốn mô hình. Giao diện hiển thị ba lớp có xác suất cao nhất của từng mô hình, ảnh 28×28 mà mô hình thật sự nhận, thời gian dự đoán và báo các mô hình đồng thuận hay bất đồng (Hình 8). Các mô hình đồng thuận không đồng nghĩa với dự đoán đúng; hình minh hoạ chỉ phục vụ kiểm thử chức năng, không thay cho đánh giá định lượng.



(a) Giao diện demo.



(b) Các mô hình đồng thuận.



(c) Các mô hình bất đồng.

Hình 8: Demo nhận dạng chữ số vẽ tay.

8 Kết luận và hướng phát triển

Kết luận. Nhóm đã tự cài đặt, kiểm chứng và so sánh bốn bộ phân lớp trên MNIST. MLP 784-256-128-10 tốt nhất với accuracy test 98.09% (F1 macro 0.9808), tiếp theo là Random Forest (96.45%), Logistic Regression (92.13%) và Multinomial Naive Bayes (83.64%). Hai mô hình tuyến tính underfit vì biên quyết định tuyến tính; hai mô hình phi tuyến chính xác hơn nhưng khoảng cách train–test lớn hơn. Trên chữ viết của 30 người Việt, accuracy của MLP giảm từ 98.13% xuống 89.35%; bổ sung một ít dữ liệu đúng miền đưa MLP lên 94.56% và Random Forest từ 68.79% lên 87.89%, hiệu quả hơn augmentation. Augmentation giúp MLP giữ 77.91% accuracy khi ảnh bị dịch 3 pixel, so với 20.19% khi không tăng cường.

Hướng phát triển. Thử mạng tích chập (CNN) để khai thác cấu trúc không gian và tính bất biến tịnh tiến; thu thập thêm chữ viết tay người Việt đa dạng hơn về người viết và kiểu chữ; vector hoá việc duyệt cây và huấn luyện song song để Random Forest nhanh hơn.

Tài liệu

[1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, 2006. Mô hình tuyến tính, softmax, mạng nơ-ron, backpropagation.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. MLP, tối ưu hóa.

[3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, second edition, 2009. Cây, boosting, Random Forest.

[4] Stanford CS231n. Lecture notes: Linear classification, optimization, backpropagation, neural networks. Stanford University Course Notes, 2015. Tài liệu thực hành về cài đặt thuật toán từ đầu và kiểm tra gradient.

A Phân công công việc và khai báo sử dụng AI

A.1 Bảng phân công công việc và tỷ lệ đóng góp

Bảng dưới đây trình bày nhiệm vụ được phân công và tỷ lệ đóng góp của từng thành viên trong quá trình thực hiện đề tài.

STT	**Thành viên**	**Công việc thực hiện**	**Đóng góp (%)**
1	Nguyễn Thị Huệ Giang	Xây dựng phương pháp Multinomial NB, Random Forest, thu thập và xử lý phiếu chữ viết tay	100%
2	Lâm Hà Quỳnh Như	Tải, tiền xử lý, chia tập dữ liệu, xây dựng phương pháp MLP, thu thập và xử lý phiếu chữ viết tay	100%
3	Hồ Việt Thuận Phú	Logistic Regression, thí nghiệm độ bền (augmentation),tạo tập dữ liệu Việt, xây dựng khung demo, thực hiện các đánh giá	100%
		Tổng cộng	**100%**

Bảng 10: Phân công công việc và tỷ lệ đóng góp của các thành viên.

A.2 Khai báo sử dụng trí tuệ nhân tạo (AI)

Trong quá trình thực hiện đề tài, nhóm có sử dụng công cụ trí tuệ nhân tạo (AI) để hỗ trợ giải thích thuật toán, tham khảo cách triển khai chương trình, phát hiện và sửa lỗi mã nguồn, cũng như chỉnh sửa nội dung báo cáo.

Các kết quả do AI đề xuất được nhóm xem xét, kiểm tra và điều chỉnh trước khi sử dụng. Nhóm chịu trách nhiệm về tính chính xác của nội dung, mã nguồn và kết quả thực nghiệm được trình bày trong báo cáo.

Công cụ AI	**Mục đích sử dụng**	**Phạm vi sử dụng**
ChatGPT	Hỗ trợ giải thích, gợi ý mã nguồn và sửa lỗi báo cáo	Tham khảo trong quá trình viết báo cáo
Claude	• Soạn tài liệu hướng dẫn: kiến thức nền, công thức, ví dụ tính tay • Gợi ý mã nguồn khung: `metrics.py`, `vn_digits.py`, `experiments.py`, `app.py`, notebook `main.ipynb` • Thiết kế phiếu thu thập chữ số viết tay • Giải thích và sửa lỗi khi chạy	• Giải thích thuật toán và cách kiểm chứng (gradient check, đối chiếu scikit-learn) • Rà soát code nhóm tự viết

Bảng 11: Khai báo công cụ AI được sử dụng trong đề tài.